

```

1  import java.util.Scanner;
2
3  public class StackTest {
4      public static void main(String[] args) {
5          Stack<Integer> stkInt = buildIntStack();
6
7          System.out.println(stkInt.toString());
8          System.out.println("sum stack " + sumStk(stkInt));
9          System.out.println("stk size "+ size(stkInt));
10
11         Stack<Character> stkCh = buildChStack();
12         System.out.println("is a in stk " + exist(stkCh, 'a'));
13         Stack<Item> stkItem = intStkToItem(stkCh);
14         System.out.println(stkItem.toString());
15     }
16
17     public static Stack<Character> buildChStack(){
18         Stack<Character> stk = new Stack<Character>();
19         System.out.print ("Enter a sequence of characters: ");
20         Scanner in = new Scanner (System.in);
21         in.useDelimiter ("");
22         char ch = in.next().charAt(0);
23         while (ch != '\n')
24         {
25             stk.push(ch);
26             ch = in.next().charAt(0);
27         }
28         return stk;
29     }
30
31     public static Stack<Integer> buildIntStack(){
32         Stack<Integer> stk = new Stack<Integer>();
33         System.out.print ("Enter a sequence of numbers: ");
34         Scanner in = new Scanner (System.in);
35         in.useDelimiter ("");
36         char ch = in.next().charAt(0);
37         while (ch != '\n')
38         {
39             stk.push(Character.getNumericValue(ch));
40             ch = in.next().charAt(0);
41         }
42         return stk;
43     }
44
45     // sums the values of the stack
46     public static int sumStk(Stack<Integer> stk) {
47         int sum = 0, x;
48         Stack<Integer> sTemp = new Stack<Integer>();
49         while (!stk.isEmpty()) {
50             x = stk.pop();
51             sum += x;
52             sTemp.push(x);
53     }

```

```

54     while (!sTemp.isEmpty()) {
55         stk.push(sTemp.pop());
56     }
57     return sum;
58 }
59
60 public static int size(Stack<Integer> stk) {
61     int counter = 0;
62     Stack<Integer> sTemp = new Stack<>();
63     while (!stk.isEmpty()) {
64         sTemp.push(stk.pop());
65         counter++;
66     }
67     while (!sTemp.isEmpty()) {
68         stk.push(sTemp.pop());
69     }
70     return counter;
71 }
72
73 public static boolean exist(Stack<Character> stk, char x) {
74     Stack<Character> sTemp = new Stack<>();
75     boolean found = false;
76     while (!stk.isEmpty() && stk.top() != x) {
77         sTemp.push(stk.pop());
78     }
79     if (!stk.isEmpty()) {
80         found = true;
81     }
82     while (!sTemp.isEmpty()) {
83         stk.push(sTemp.pop());
84     }
85     return found;
86 }
87
88 public static void popChar(Stack<Character> stk, char x) {
89     Stack<Character> sTemp = new Stack<>();
90     while (!stk.isEmpty() && stk.top() != x) {
91         sTemp.push(stk.pop());
92     }
93     if (!stk.isEmpty()) {
94         stk.pop();
95     }
96     while (!sTemp.isEmpty()) {
97         stk.push(sTemp.pop());
98     }
99 }
100
101 public static Stack<Integer> clone(Stack<Integer> stk)
102 {
103     Stack<Integer> sTemp = new Stack<>();
104     while (!stk.isEmpty())
105         sTemp.push(stk.pop());
106

```

```

107     Stack<Integer> sCopy = new Stack<>();
108     while (!sTemp.isEmpty())
109     {
110         sCopy.push(sTemp.top());
111         stk.push(sTemp.pop());
112     }
113     return sCopy;
114 }
115
116 public static Stack<Item> intStkToItem(Stack<Character> stk) {
117     Stack<Item> sTemp = new Stack<>();
118     int counter = 0;
119     while (!stk.isEmpty()) {
120         counter = 0;
121         char top = stk.top();
122         while (exist(stk, top)) {
123             popChar(stk, top);
124             counter++;
125         }
126         sTemp.push(new Item(top, counter));
127     }
128     return sTemp;
129 }
130
131 }
132
133
134
135 /*
136 Enter a sequence of numbers: 324623
137 [3, 2, 6, 4, 2, 3]
138 sum stack 20
139 stk size 6
140 Enter a sequence of characters: abcdasda
141 [(b, 1), (c, 1), (s, 1), (d, 3), (a, 3)]
142 */

```